8c. Type Stability with Scalars and Vectors

Martin Alfaro
PhD in Economics

INTRODUCTION

The previous section has defined type stability, along with approaches to checking whether the property holds. The formal definition of a type-stable function is that the function's output type can be inferred from its argument types. In practice, however, we often rely on a more stringent definition, which requires that the compiler can infer a single concrete type for each expression within the function body. This property guarantees that every operation is specialized, resulting in optimal performance. [1]

In this section, we start the analysis of type stability for specific objects. We cover in particular the case of scalars and vectors, providing practical guidance for achieving type stability with them.

TYPES OF SCALARS AND VECTORS

The notion of type stability applied to scalars is straightforward. It demands operations to be performed on variables with the same concrete type (e.g., `Float64`, `Int64`, `Bool`). Likewise, type stability for vectors requires that their *elements* have a concrete type.

The following table identifies types for scalars and vectors satisfying this property.

Objects Whose Elements Have Concrete Types

Scalars	Vectors
`Int`	`Vector{Int}`
`Int64`	`Vector{Int64}`
`Float64`	`Vector{Float64}`
`Bool`	`BitVector`

Note: `Int` defaults to `Int64` or `Int32`, depending on the CPU architecture.

Next, we'll delve into type stability in scalars and vectors, considering each case separately.

TYPE STABILITY WITH SCALARS

To make the definition of type stability for scalars operational, let's revisit some concepts about types. Recall that only concrete types like `Int64` or `Float64` can be instantiated, while abstract types like `Any` or `Number` can't. Specifically, when a value is instantiated, we mean that it ultimately has a single concrete type. Abstract types, by contrast, never hold values directly. Their role is purely organizational: they describe sets of possible concrete types and help structure the type hierarchy.

This distinction explains why a type annotation such as `x::Number` shouldn't be read as `x` having type `Number`. Rather, it constrains `x` to values whose concrete types are subtypes of `Number`. At runtime, though, `x` must always have a concrete type. For instance, after evaluating `x::Number = 2`, the variable `x` contains the value 2, whose concrete type is `Int64`.

With this in mind, we can discuss how type instability arises. A common source of instability is mixing values of different types, such as combintations of `Int64` with `Float64`. However, mixing types doesn't automatically imply type instability. This leads us to define the concepts of type promotion and conversion.

TYPE PROMOTION AND CONVERSION

Julia employs various mechanisms to handle cases combining `Int64` and `Float64`. The first one is part of a concept known as **type promotion**, which converts dissimilar types to a common one whenever possible. The second one emerges when variables are type-annotated, in which case Julia engages in **type conversion**. By transforming values to the respective type declared, this feature also prevents the mix of types.

Both mechanisms are illustrated below.

TYPE PROMOTION

```
foo(x,y)     = x * y

x1           = 2
y1           = 0.5

output        = foo(x1,y1)          # type stable: mixing `Int64` and `Float64` results in
`Float64`
```

```
julia> output
1.0
```

TYPE CONVERSION

```
foo(x,y)     = x * y

x2::Float64 = 2                # this is converted to `2.0`
y2          = 0.5

output      = foo(x2,y2)         # type stable: `x` and `y` are `Float64`, so output type is
predictable
```

```
julia> output
1.0
```

In the first tab, the output type depends on the argument type. However, in all cases the output type can be predicted, since mixing `Int64` and `Float64` results in `Float64` due to automatic type promotion. As for the second tab, Julia transforms the value of `x2` to make it consistent with the type-annotation declared. Consequently, `x * y` is computed as the product of two values with type `Float64`.

TYPE INSTABILITY WITH SCALARS

While type promotion and conversion can handle certain situations, they certainly don't cover all cases. One such scenario is when a scalar value depends on a conditional statement, with each branch returning a value of a different type. In this situation, since the compiler only considers the types and not values, it can't determine which branch is relevant for the function call. As a result, it'll generate code that accommodates both possibilities, as it happens in the following example.

INT64

```
function foo(x,y)
    a = (x > y) ?  x  :  y

    [a * i for i in 1:100_000]
end

foo(1, 2)            # type stable   -> `a * i` is always `Int64`
```

```
julia> @btime foo(1,2)
  17.608 µs (3 allocations: 781.312 KiB)
```

MIXING INT64 AND FLOAT64

```
function foo(x,y)
    a = (x > y) ?  x  :  y

    [a * i for i in 1:100_000]
end

foo(1, 2.5)          # type UNSTABLE -> `a * i` is either `Int64` or `Float64`
```

```
julia> @btime foo(1,2.5)
  45.474 µs (3 allocations: 781.312 KiB)
```

In the example, type instability will inevitably arise if `x` and `y` have different types. Note that type promotion is of no help here. The reason is that this mechanism only ensures that `a * i` will be converted to `Float64` if `a` is `Float64`, considering that `i` is `Int64`. However, the compiler also needs to consider the possibility that `a` could be `Int64`, in which case `a * i` would be `Int64`.

Given this ambiguity, the method instance created must be capable of handling both scenarios. Then, during runtime, Julia will gather more information to disambiguate the situation, and select the relevant computation implementation.

TYPE STABILITY WITH VECTORS

Vectors in Julia are formally defined as collections of elements sharing a homogeneous type. Since operations based on vectors ultimately handle individual elements, type stability is contingent on whether the type of their elements is concrete.

This is why it's important to distinguish between the type of the object and the type of its elements. In particular, note that vectors having elements with a concrete type are themselves concrete, but elements with abstract types will still give rise to vectors with concrete types. This is clearly observed with `Vector{Any}`, a concrete type comprising elements with the abstract type `Any`.

Before the analysis of specific scenarios, we start by considering type promotion and conversion applied to vectors.

TYPE PROMOTION AND CONVERSION

By definition, vectors require all their elements to share the same type. This means that if you mix elements with disparate types, such as `String` and `Int64`, Julia will infer the vector's type as `Vector{Any}`. Despite this, there are cases where elements can be converted to a common type, such as when mixing `Float64` and `Int64`.

The following example shows this mechanism in an assignment, where the vector is not type annotated. In this case, all elements are converted to the most general type among the values included.

VECTOR ASSIGNMENT - CONVERSION 1

```
x = [1, 2, 2.5]      # automatic conversion to `Vector{Float64}`
```

```
julia> x
3-element Vector{Float64}:
 1.0
 2.0
 2.5
```

VECTOR ASSIGNMENT - CONVERSION 2

```
y = [1, 2.0, 3.0]    # automatic conversion to `Vector{Float64}`
```

```
julia> y
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

When assignments are instead declared with type-annotations and values are of different types, Julia will attempt to perform a conversion. If possible, this ensures that the assigned values conform to the declared type.

VECTOR ASSIGNMENT - NO CONVERSION 1

```
v1                = [1, 2.0, 3.0]     # automatic conversion to `Vector{Float64}`

w1::Vector{Int64}  = v1                # conversion to `Vector{Int64}`
```

```
julia> w1
3-element Vector{Int64}:
 1
 2
 3
```

VECTOR ASSIGNMENT - NO CONVERSION 2

```
v2                = [1, 2, 2.5]       # automatic conversion to `Vector{Float64}`

w2::Vector{Number} = v2                # `w2` is still `Vector{Number}`
```

```
julia> w2
3-element Vector{Number}:
 1.0
 2.0
 2.5
```

TYPE INSTABILITY

When evaluating type stability with vectors, two forms of operations must be considered. The first one involves operations that manipulate individual elements `x[i]`. This scenario is analogous to the case of scalars, and therefore type stability follows the same rules.

The second scenario involves functions operating on the entire vector. In this case, type stability requires that vectors have elements with a concrete type. Note that this condition isn't sufficient to guarantee type stability, which ultimately depends on how the function implements the operation executed.

Nevertheless, packages tend to provide optimized versions of functions. Consequently, functions are typically type stable when users provide vectors with elements of a concrete type. For instance, this is illustrated below by the function `sum`, which adds all elements in a vector.

VECTOR{INT}

```
z1::Vector{Int}     = [1, 2, 3]

sum(z1)             # type stable
```

VECTOR{INT64}

```
z2::Vector{Int64}   = [1, 2, 3]

sum(z2)             # type stable
```

VECTOR{FLOAT64}

```
z3::Vector{Float64} = [1, 2, 3]

sum(z3)             # type stable
```

BITVECTOR

```
z4::BitVector       = [true, false, true]

sum(z4)             # type stable
```

In contrast, the following vectors have elements with abstract types, which result in type instability.

VECTOR{NUMBER}

```
z5::Vector{Number} = [1, 2, 3]

sum(z5)                 # type UNSTABLE -> `sum` must consider all possible subtypes of
`Number`
```

VECTOR{ANY}

```
z6::Vector{Any}    = [1, 2, 3]

sum(z6)             # type UNSTABLE -> `sum` must consider all possible subtypes of `Any`
```

FOOTNOTES

[1.] Nevertheless, simply demanding that the output's type can be inferred from the input types already offers benefits. In particular, it ensures that type instability won't be propagated when the function is called in other operations.